

Measuring Internal Product Attributes: Structural Complexity of mindPal

1. CONTROL FLOW STRUCTURE

MindPal's server.js contains clearly defined control flow structures that can be modelled using a Control Flow Graph (CFG). The main chat route /api/chat follows a sequence of predicate nodes that branch execution based on conditions.

The control flow of the chat route follows this logical path. First the server checks whether the messages array exists and is a valid array. If this condition is false execution branches to return a 400 error response and the function terminates. If the condition is true execution continues to the next predicate node which checks whether the GROQ_API_KEY environment variable is set and is not the placeholder string. If this condition is false execution branches to return a 500 error response. If the condition is true execution continues to the try block where the Groq API fetch request is made. Inside the try block another predicate checks whether the HTTP response was successful using response.ok. If false it branches to return the Groq error. If true it extracts the reply and returns it. The catch block handles any network or server-level failure as a separate terminal branch.

Using the code-based cyclomatic complexity formula $v(G) = 1 + d$ where d is the number of predicate nodes (decision points), the /api/chat route has the following decision points. The check for missing messages array counts as one predicate. The check for missing API key counts as one predicate. The check for response.ok counts as one predicate. The try/catch structure counts as one predicate. This gives $v(G) = 1 + 4 = 5$, meaning there are five linearly independent paths through the route and a minimum of five test cases would be required to achieve full path coverage of this function.

The startup diagnostic block in server.js adds one additional predicate node checking whether the API key equals the placeholder string, contributing to the overall cyclomatic complexity of the file.

2. DEPTH OF NESTING

The server.js chat route demonstrates controlled depth of nesting that keeps complexity manageable. The outer structure is the app.post route handler which is nesting level one. Inside this sits the first if statement checking the messages array at nesting level two. The try block sits at nesting level two. Inside the try block the if statement checking response.ok sits at nesting level three. Inside that conditional the extraction of the reply using data.choices map and filter sits at nesting level four.

Using the depth of nesting formula $n(F(F1, F2...Fk)) = 1 + \max(n(F1), n(F2)...)$, the maximum depth of nesting in the chat route is four levels. According to the course framework, smaller depth of nesting indicates less complexity in coding and testing. A depth of four is acceptable for a production API route of this nature and does not indicate problematic structural complexity.

The frontend JavaScript in index.html maintains a shallower nesting depth. The ask() async function contains a try/catch at level one with a single if statement checking data.error at level two, giving a maximum nesting depth of two in the frontend request handler. This keeps the client-side logic straightforward and easy to test.

3. CYCLOMATIC COMPLEXITY

The full system cyclomatic complexity is calculated by summing the cyclomatic complexity of each component, following the CBS code complexity formula $v(G) = \text{sum of } v(C_i) \text{ for all components}$.

Component: /api/chat route in server.js. Decision points: messages validation check, API key validation check, response.ok check, try/catch boundary. $v(C1) = 1 + 4 = 5$.

Component: /api/check-ollama route (health check). Decision points: response parse try/catch, hasModel check. $v(C2) = 1 + 2 = 3$.

Component: ask() function in index.html. Decision points: busy guard check, empty input check, data.error check, try/catch boundary. $v(C3) = 1 + 4 = 5$.

Component: setMode() function in index.html. Decision points: check whether conversation history length is greater than zero before appending mode-change message. $v(C4) = 1 + 1 = 2$.

Component: triggerTool() function in index.html. Decision points: check whether tool equals mood_check to branch between showing widget and sending API prompt. $v(C5) = 1 + 1 = 2$.

Total system cyclomatic complexity: $v(G) = 5 + 3 + 5 + 2 + 2 = 17$.

According to McCabe's guidelines, a cyclomatic complexity of 10 or below per module indicates a well-structured, easily testable component. The two most complex components each score 5, which is well within the recommended threshold. The overall system score of 17 across five components reflects a system of moderate, manageable complexity appropriate for a single-developer project.

4. COHESION

MindPal's architecture demonstrates high cohesion across its components, following the principle that internal relationships within a module should significantly outweigh external relationships.

The server.js module performs a single unified purpose: receiving chat requests from the frontend, calling the Groq API with the appropriate system prompt, and returning the response. All functions within server.js serve this purpose directly. The SYSTEM_PROMPTS object, the route handlers, the startup diagnostics, and the error handling blocks all contribute to the same task. This is functional cohesion, which is the strongest form of cohesion in the classification hierarchy.

Using the CBS cohesion formula $CH(C_i) = R_{\text{internal}} / (R_{\text{internal}} + R_{\text{external}})$, the server.js component has high internal relationship density. The route handler references the SYSTEM_PROMPTS object internally, the startup block references process.env internally, and the error handler references the res object internally. The only external relationship is the outgoing HTTP call to the Groq API. This gives a high cohesion ratio for the server component.

The public/index.html frontend module is similarly cohesive. All JavaScript functions within it serve the single purpose of managing the chat interface: sending messages, displaying responses, switching modes, showing tools, and updating the session timer. There are no unrelated functions present in the file. The only external relationship is the fetch call to /api/chat on the local server.

The SYSTEM_PROMPTS object in server.js is a particularly strong example of communicative cohesion as all nine therapy mode entries operate on the same data structure (the prompt string) and are accessed by the same mechanism (key lookup by mode name), making the module's internal components tightly related in both purpose and data.

5. COUPLING

MindPal is designed to achieve loose coupling between its components, following the course principle that loose coupling reduces interdependence and improves maintainability.

The relationship between the frontend (index.html) and the backend (server.js) is data coupling, which is the loosest form of coupling in the classification hierarchy (R1). The frontend sends a JSON object containing messages and mode, and the backend returns a JSON object containing reply or error. Neither component references the internal structure of the other. The frontend has no knowledge of how the backend calls the Groq API, and the backend has no knowledge of how the frontend displays the response.

The relationship between server.js and the Groq API is also data coupling (R1). The server constructs a standard JSON request body and receives a standard JSON response. No internal Groq structures are referenced directly. If the Groq API endpoint or model name changes, only two lines in server.js require modification with no changes needed anywhere else in the system.

The .env file is decoupled from both the frontend and the Groq API entirely. It communicates its single value (GROQ_API_KEY) to server.js only through the dotenv library's process.env interface, which is a form of data coupling with no stamp or control coupling present.

Using the CBS coupling formula $CP(C_i) = R_{\text{external}} / (R_{\text{internal}} + R_{\text{external}})$, the server.js component has a low coupling ratio because the majority of its relationships are internal (route handling logic, prompt lookup, error formatting) and it has only one external dependency which is the outgoing Groq API call. The frontend has similarly low coupling with only one external relationship to the server's /api/chat endpoint.

There is no content coupling (R4) anywhere in the system. The frontend does not access or modify any internal data of the backend, and the backend does not access or modify any internal

state of the frontend. This is consistent with the course objective of achieving loose coupling throughout the architecture.

6. INFORMATION FLOW MEASURES

Using the Henry-Kafura Information Flow Complexity measure $IFC(M) = \text{fan-in}(M)^2 \times \text{fan-out}(M)$, the information flow of the central `server.js` chat route can be characterised as follows.

Fan-in of the `/api/chat` route: the route receives information from the frontend POST request body (one flow), from the `process.env.GROQ_API_KEY` environment variable (one flow), and from the `SYSTEM_PROMPTS` object lookup (one flow). Fan-in = 3.

Fan-out of the `/api/chat` route: the route sends information to the Groq API via the outgoing fetch request (one flow), and returns the JSON response to the frontend (one flow). Fan-out = 2.

$IFC(/api/chat) = (3^2) \times (2^2) = 9 \times 4 = 36$.

This is a moderate information flow complexity score. The Henry-Kafura measure indicates that the `/api/chat` route is the most informationally complex function in the system, which is expected as it is the central integration point between the user interface, the configuration layer, the prompt library, and the external API. All other functions in the system have lower fan-in and fan-out values. The `ask()` function in the frontend has a fan-in of 2 (user input and conversation history) and a fan-out of 2 (server POST request and DOM update), giving $IFC = 4 \times 4 = 16$.

7. MORPHOLOGICAL MEASURES

Representing MindPal's architecture as a graph $S = \{N, R\}$ where each node is a subsystem and each edge is a relationship between subsystems, the following morphological measurements apply.

Nodes (N): frontend interface (`index.html`), backend server (`server.js`), environment configuration (`.env`), `SYSTEM_PROMPTS` library (embedded in `server.js`), Groq API (external), browser DOM (rendered output). This gives 6 nodes.

Edges (R): frontend to server via `/api/chat` POST, server to `.env` via `dotenv` `process.env`, server to `SYSTEM_PROMPTS` via key lookup, server to Groq API via HTTP fetch, Groq API to server via HTTP response, server to frontend via JSON reply, frontend to browser DOM via DOM manipulation. This gives 7 edges.

Size: 6 nodes and 7 edges. Depth: The longest path from input to output is frontend to server to Groq API to server to frontend, giving a depth of 4. Width: The widest level is the server layer which connects to `.env`, `SYSTEM_PROMPTS`, Groq API, and frontend simultaneously, giving a width of 4. Edge-to-node ratio: $7 \div 6 = 1.17$, indicating a near-tree structure with minimal cross-connections.

Using the tree impurity formula $m(G) = (e \text{ minus } n \text{ plus } 1) \text{ divided by } ((n \text{ minus } 1) \text{ multiplied by } (n \text{ minus } 2) \text{ divided by } 2)$, the low edge-to-node ratio of 1.17 confirms that the architecture is close to a tree structure. A tree has an edge-to-node ratio of exactly 1.0 for a connected graph. MindPal's ratio of 1.17 indicates very low tree impurity, meaning the design is hierarchical, minimally cross-coupled, and easy to reason about structurally.

8. DATA STRUCTURE COMPLEXITY

There is always a trade-off between control-flow complexity and data structure complexity, as noted in the course material. MindPal's moderate cyclomatic complexity of 5 per major component is balanced by a moderately complex data structure in the conversation history array.

The primary data structure in MindPal is the conversation history array maintained in the frontend JavaScript. Each element is an object with two string fields: role and content. Using the data structure complexity framework from the course:

String fields per message object: 2 strings multiplied by a cost of 2 = 4 per object. Maximum practical session length: approximately 20 message exchanges = 20 objects. Estimated data structure complexity: 20 multiplied by 4 = 80 units.

The SYSTEM_PROMPTS object contains 9 string entries of substantial length (each prompt averaging approximately 1500 characters). Each entry is a string with a string cost of 2 per entry, giving 9 multiplied by 2 = 18 units for the prompt data structure, plus the character volume.

This confirms the trade-off described in the course. The server.js has moderate cyclomatic complexity ($v = 5$ for the main route) because the data structures do the heavy lifting. The therapy logic is encoded in the SYSTEM_PROMPTS strings rather than in branching conditional logic. A design that used if/else chains to select therapeutic language would have far higher cyclomatic complexity but simpler data structures. MindPal deliberately trades lower control-flow complexity for richer data structure complexity, resulting in a more maintainable and extensible system.